CA

Wednesday, September 9, 2026 3:46 PM

Printout

Wednesday, October 21, 2026 9:50 PM

CSC 402 - Activity 10 - Name:

Remark 1. In the notes/videos I have primarily stuck with the classic sigmoid/logistic activation function

$$\sigma(x) = \frac{1}{1+e^{-x}}.$$

https://www.desmos.com/calculator/6gda2dg5vb This function is commonly used for models where we have to predict the probability as an output. Since probability of anything exists only between 0 and 1, the Sigmoid nicely fits that bill. Moreover, this function is differentiable and provides a smooth gradient, i.e., preventing jumps in output values. In particular, it has a very nice derivative shape, $\sigma'(x) = \sigma(x)(1-\sigma(x))$. If you were to graph this derivative you would see there is a large drop-off away from $[-3,3]$ in the derivative. This produces gradient values that will end up being very small. Remember, as the gradient value approaches zero, the network ceases to learn (stabilizes) and can suffer from the vanishing gradient problem.

Spoiler alert – there are many types of activation functions and we can use different functions for the input layer, inner layers, and output layer. We will further explore other activation functions and their applications in later sections/activities.

Problem 1. The Hyperbolic Tangent function is defined as

$$f(x) = \tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}.$$

https://www.desmos.com/calculator/edo1f4iuqi So, why is this function nice?

a. This function takes on values between -1 and 1 and is zero centered. Show that f is zero centered/symmetric about the origin, that is $f(-x) = -f(x)$.

$f(-x) = \frac{e^{-x}-e^{-(-x)}}{e^{-x}+e^{-(-x)}} = -\left[\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}\right] = -f(x)$

Note: This fact makes using the Hyperbolic Tangent function useful in hidden layers of a neural network. Since the values lie between -1 and 1 the mean for the hidden layer comes out to be 0 or very close to it. This centering the data make learning for the next layer easier.

b. Its derivative has a nice shape/formula. Show that $f'(x) = 1-\tanh^2(x) = 1-(f(x))^2$.

$f'(x) = \frac{\frac{d}{dx}(e^{x}-e^{-x})(e^{x}+e^{-x}) - (e^{x}+e^{-x})\frac{d}{dx}(e^{x}-e^{-x})}{(e^{x}+e^{-x})^2}$

$= \frac{(e^{x}+e^{-x})^2 - (e^{x}+e^{-x})^2}{(e^{x}+e^{-x})^2} = 1 - \left[\frac{e^{x}+e^{-x}}{e^{x}+e^{-x}}\right]^2 = 1-\tanh^2(x) = 1-(f(x))^2$

Note: Although both sigmoid and tanh face vanishing gradient issue, tanh is zero centered, and the gradients are not restricted to move in a certain direction. Therefore, in practice, tanh nonlinearity is sometimes preferred to sigmoid nonlinearity.

Hint: There are many ways you can show the above result. If you have played with Hyperbolic Trig functions before you may recognize/remember that $\tanh(x) = \frac{\sinh(x)}{\cosh(x)}$ and $\operatorname{sech}^2(x) = 1-\tanh^2(x)$. If not, you can show the derivative simply through the exponentials!

Remark 2. Another page and another activation function! Next up, the Rectified Linear Unit, ReLU. The ReLU function takes on the value of 0 for negative input and the value of x for non-negative input. Formally, we have

$$f(x) = \max(0, x) = \begin{cases} 0 & \text{if} \quad x < 0 \\ x & \text{if} \quad x \geq 0 \end{cases}$$

Since this function zeros out any negative input the ReLU function does not activate all the neurons at the same time. The neurons will be deactivated if the pre-activation input is less than 0. Additionally, we have the following advantages

a. As many of the neurons will be deactivated, the ReLU function is more computationally efficient when compared to the sigmoid and hyperbolic tangent functions.
b. ReLU accelerates the convergence of gradient descent towards the global minimum of the loss function due to its linear, non-saturating property.

There are some big issues with the classic ReLU function. Namely, its derivative is simply

$$f'(x) = \begin{cases} 0 & \text{if} \quad x < 0 \\ 1 & \text{if} \quad x > 0 \end{cases}.$$

As the negative half is zeroed out the gradient will also be zero. Thus, during back-propagation the weights and bias values for some neurons are not updates. This can create dead neurons which are never updated.

Problem 2. The Exponential Linear Units, or ELU for short, is a variant of ReLU that adjusts the slope of the negative apart of the function. ELU uses a log curve to define the negative values. Formally, ELU is defined as

$$f(x) = \begin{cases} \alpha(e^x - 1) & \text{if} \quad x < 0 \\ x & \text{if} \quad x \geq 0 \end{cases}$$

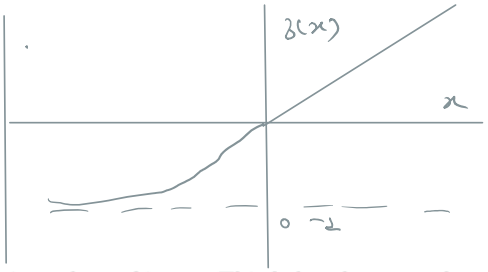
where α is a positive constant. https://www.desmos.com/calculator/ekkkm2gzyq Typically the default value for α is 1 as this ensures differentiability at 0. ELU is a good alternative for ReLU as it has the following advantages.

a. ELU becomes smooth slowly until its output equals $-\alpha$. Sketch a graph of ELU for $\alpha = 1$ and show/confirm that $f(x) \geq -\alpha$ for all x and the limit as $x \to -\infty$ is $-\alpha$.

For $x<0$, $e^x > 0$, $e^x - 1 > -1$, $f(x) > -\alpha$

For $x \geq 0$, $f(x) = x \geq 0$, $f(x) \geq 0 > -\alpha$, $f(x) \geq -\alpha$

$\lim_{x \to -\infty} f(x) = \lim_{x \to -\infty} (e^x - 1) = -1 = -\alpha$



b. Avoid the dead ReLU problem by introducing a log curve for negative values of input. This helps the network nudge weights and bias values in the right direction. Show that the derivative of ELU is

$$f'(x) = \begin{cases} \alpha e^x & \text{if} \quad x < 0 \\ 1 & \text{if} \quad x > 0 \end{cases} = \begin{cases} f(x) + \alpha & \text{if} \quad x < 0 \\ 1 & \text{if} \quad x > 0 \end{cases}.$$

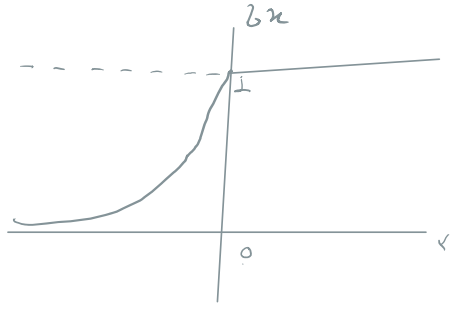
Then sketch a graph of the derivative of ELU for $\alpha = 1$.

For $x<0$, $\frac{d}{dx}\alpha(e^x - 1) = \alpha e^x$

For $x>0$, $\frac{dx}{dx} = 1$

So $f'(x) = \begin{cases} \alpha e^x & x < 0 \\ 1 & x > 0 \end{cases}$

For $x<0$, $f(x) = \alpha(e^x - 1)$, $f(x) = e^x \alpha - \alpha$, $f(x) + \alpha = \alpha e^x$, $f'(x) = f(x) + \alpha$



There are some limitations of the ELU function.

a. The introduction of the exponential function does increase the computation time.
b. We need to preload the value of α as there is no learning the value of α within learning algorithms

Problem 3. Next up, the Sigmoid Linear Unit (SiLU) or Swish. SiLU is defined as

$$f(x) = \frac{x}{1+e^{-x}} = x\sigma(x).$$

Swish matches or outperforms ReLU activation function on deep networks when applied to challenging domains such as image classification.

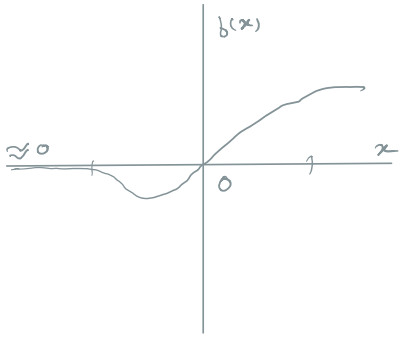
a. Sketch a graph of $f(x)$ and confirm $f(x) \approx x$ for $x > 0$ and $f(x) \approx 0$ for $x < 0$. (Does the name Swish make sense?)

For $x>0$
$f(x) = \frac{x}{1+0}$ [e^{-x} will be 0]
$\Rightarrow f(x) \approx x$

For $x<0$
$f(x) = \frac{x}{1+\infty}$
$= 0$
$f(a) \approx 0$

For $x=0$
$f(0) = \frac{0}{1+e^0}$
$= 0$

Yes Swish makes sense



b. Show that $f'(x) = \frac{1+e^{-x}+xe^{-x}}{(1+e^{-x})^2} = \sigma(x)(1+x-x\sigma(x))$. [Hint: Product Rule $(ab)' = ab' + ba'$]

$$f'(x) = \frac{(1+e^{-x})\frac{dx}{dx} - x\frac{d(1+e^{-x})}{dx}}{(1+e^{-x})^2} = \frac{1+e^{-x}+xe^{-x}}{(1+e^{-x})^2} = \frac{1+e^{-x}}{(1+e^{-x})^2} + \frac{xe^{-x}}{(1+e^{-x})^2}$$

$$= \frac{1}{(1+e^{-x})} + \frac{x}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} \quad \left[1 - \frac{1}{1+e^{-x}} = \frac{e^{-x}}{1+e^{-x}}\right]$$

$$= \sigma(x) + x\sigma(x)\left[1 - \frac{1}{1+e^{x}}\right] = \sigma(x)\left[1+x-x\sigma(x)\right]$$

Problem 4. Lastly, Softplus is a smooth approximation of ReLU. Softplus is defined as

$$f(x) = \ln(1+e^x)$$

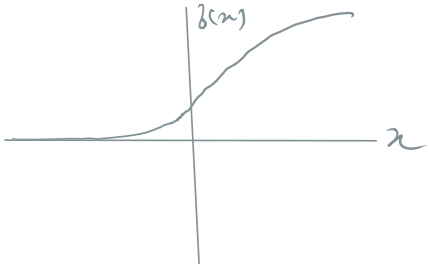
Softplus provides a smooth and differentiable alternative to the ReLU activation function, offering improved numerical stability and gradient flow in deep networks, though typically without significant gains in accuracy compared to ReLU on large-scale tasks.

a. Sketch a graph of $f(x)$ and confirm $f(x) \approx x$ for $x > 0$ and $f(x) \approx 0$ for $x < 0$.

For $x>0$
$f(x) = \ln(1+e^x)$
$= \ln(e^x)$
$= x$

For $x<0$
$f(x) = \ln(1+0)$
$= \ln(1)$
≈ 0

For $x=0$
$f(0) = \ln(1+e^0)$
$= \ln(1+1)$
$= \ln 2$
$= 0.693$



b. Determine $f'(x)$. [Hint: The natural log has a really nice derivative: $\frac{d}{dx}[\ln(x)] = \frac{1}{x}$]

$$f'(x) = \frac{d}{dx}(\ln(1+e^x))$$

$$= \frac{d}{d(1+e^x)}[\ln(1+e^x)] \cdot \frac{d}{dx}(1+e^x)$$

$$= \frac{e^x}{1+e^x}$$

Printout

Saturday, October 24, 2026 10:39 PM

CSC 402 - Activity 11 - Name:

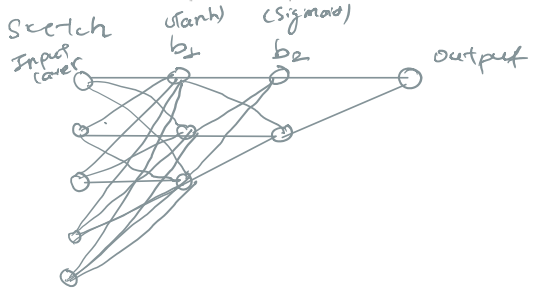
Problem 1 (Learning and Backpropagation!).

a. Consider the following training data set. Generate neural network with two hidden layers – 3 neurons with TANH activation in first hidden layer and 2 neurons with SIGMOID activation in the second hidden layer. Iterate 10000 times and record the loss and activations for each training data point. Then sketch out the network with final weights and bias values.

Input	Output	Model Output
(0, 1, 0, 1, 0)	0	0
(1, 0, 1, 0, 1)	1	1
(1, 0, 1, 1, 0)	1	1
(0, 1, 0, 0, 1)	1	1
(1, 0, 0, 1, 1)	1	1
(1, 0, 1, 1, 1)	0	1
(1, 1, 1, 0, 1)	1	1
(0, 0, 0, 0, 1)	0	0
(1, 1, 1, 1, 0)	0	0
(1, 1, 0, 0, 1)	1	1
		1,

Input	Output	Model Output (see part c)
(0, 1, 0, 1, 0)	0	0
(1, 0, 1, 0, 1)	1	1
(1, 0, 1, 1, 0)	1	1
(0, 1, 0, 0, 1)	1	1
(1, 0, 0, 1, 1)	1	1
(1, 0, 1, 1, 1)	0	1
(1, 1, 1, 0, 1)	1	1
(0, 0, 0, 0, 1)	0	0
(1, 1, 1, 1, 0)	0	0
(1, 1, 0, 0, 1)	1	1
(1, 1, 1, 1, 1)	1	1
(0, 0, 0, 0, 0)	1	0

Final loss = 0.675443



b. For the model in part b what is the final activation for $(1, 1, 1, 1, 1)$ and $(0, 0, 0, 0, 0)$?

The final activation are 1 & 0 respectively

c. Add the following data points to your training set. Then regenerate a network, iterate 10000 times, and record the loss and activations for each training data point.

Input	Output
(1, 1, 1, 1, 1)	1
(0, 0, 0, 0, 0)	1

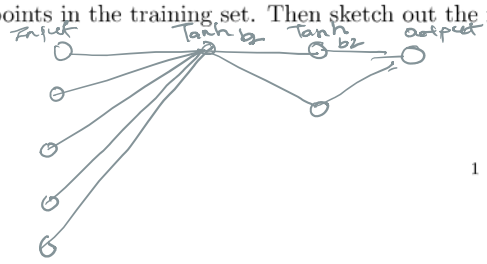
d. For the training set in part b we used a pretty large network compared to the size of input/output. (2 layers and 5 neurons) – i.e., a lot of unnecessary information.

i. Use a single neuron per layer (activations of your choice). After iterating 10000 times does the final activation match for each training pair? If there is an error, what data point caused problems?

-> No, the network doesn't classify every training example - The problematic data point is (1, 0, 1, 1, 0)

ii. Determine the fewest number of neurons that are needed to construct and train a network that aligns with all points in the training set. Then sketch out the final network below.

=>



Problem 2 (ELU). In Activity … d' RELU. For $\alpha = 1$, implement E…

$$f(x) = \begin{cases} \\ \end{cases}$$

```
# Here I wanted you to add ELU to the mix (but ended up giving it to you)
# You may not be familar with np.where(z > 0, z, np.exp(z) - 1)
# Here if the condition is true we enter the middle expression otherwise enter the last expressio
# So, it is pretty quick to implement. Check out the implentation above :-).
def elu(z):
    return np.where(z > 0, z, np.exp(z) - 1)

def delu(a):
    return np.where(a > 0, 1, a + 1)
```

Problem 3 (Of course we are are doing XOR again!). We can treat the truth table for the XOR gate as a set of training data (not really training when it is the whole set, but we can still call it that).

a.

Input	XOR (Needed Output)	Model Output (i)	Model Output (ii)	Model Output (iii)
(0,0)	0	0	1	1
(0,1)	1	1	0	1
(1,0)	1	1	1	0
(1,1)	0	0	0	0

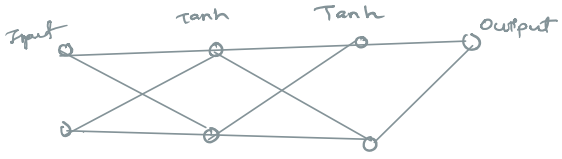
For each of the following: Generate an initial model (based on the requirements), iterate 10000 times and record the output of the training data set in the table above. For one that 'works' sketch out the network with final weights and bias values.

i. Two sigmoid neurons in both hidden layers and sigmoid one neuron in the output layer.

ii. Two sigmoid neurons in the first hidden layer and one sigmoid neuron in the second hidden layer and output layer.

iii. One sigmoid neuron in all three layers.

i, works



b. Use SIGMOID, TANH, ELU, or RELU for hidden layers (your choice!)

Input	XOR (Needed Output)	Model Output (i)	Model Output (ii)	Model Output (iii)
(0,0)	0	0	0	0
(0,1)	1	1	0	0
(1,0)	1	1	1	0
(1,1)	0	0	0	0

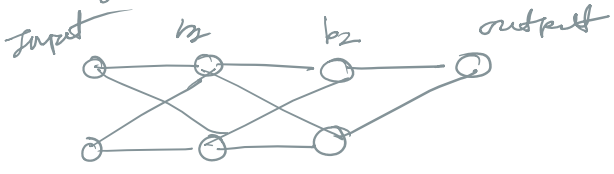
For each of the following: Generate an initial model (based on the requirements), iterate 10000 times and record the output of the training data set in the table above. For one that 'works' sketch out the network with final weights and bias values.

i. Two [SIGMOID, TANH, ELU, or RELU] neurons in both hidden layers and one sigmoid neuron in the output layer.

ii. Two [SIGMOID, TANH, ELU, or RELU] neurons in the first hidden layer, one [SIGMOID, TANH, ELU, or RELU] neuron in the second hidden layer, and one sigmoid neuron in the output layer.

iii. One [SIGMOID, TANH, ELU, or RELU] neuron in the hidden layers and one sigmoid neuron in the output.

i) Tanh, Tanh, Sigmoid



CSC 402 - Activity 12 - Name:

Definition 1 (Softmax)**.** If $z = (z_0, z_1, \ldots, z_n)$, then

$$S_i(z) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

and

$$S(z) = (S_0(z), S_1(z), \ldots S_n(z)).$$

Definition 2. We compute/define the Cross Entropy Loss (LogLoss) via

$$\mathbb{L} = Loss_{CE}(y, z) = -\sum_{i=0}^{n-1} w_i \log(S_i(z)) = -\sum_{i=0}^{n-1} w_i \log\left(\frac{e^{z_i}}{\sum_j e^{z_j}}\right)$$

where w_i is the binary indicator (0 or 1) if the label is the correct classification (one-hot encoding). Here log is typically the natural logarithm, but it can be base 2, base 10, or others.

Problem 1 (Softmax and Cross Entropy Loss!)**.** First up, let's have you play with Softmax and Cross Entropy Loss by hand. Let

$$z = (13.7, 4.2, 1.3, 3.7, 1.7).$$

a. Compute $\sum_i e^{z_i}$. (i.e., the denominator term for Softmax).

$\sum_i e^{z_i} = e^{13.7} + e^{4.2} + e^{1.3} + e^{3.7} + e^{1.7} = 891489.743$

b. Compute $S(z) = (S_0(z), S_1(z), S_2(z), S_3(z), S_4(z))$.

$S_0(z) = \frac{e^{13.7}}{\sum_i e^{z_i}} = 0.99987$ $S_2(z) = \frac{e^{1.3}}{\sum_i e^{z_i}} = 0.000004418$ $S_4(z) = \frac{e^{1.7}}{\sum_i e^{z_i}} = 0.000006145$

$S_1(z) = \frac{e^{4.2}}{\sum_i e^{z_i}} = 0.0000748$ $S_3(z) = \frac{e^{3.7}}{\sum_i e^{z_i}} = 0.0000453$

c. For $y = (1, 0, 0, 0, 0)$, compute $Loss_{CE}(y, z)$.

$Loss_{CE} = -w_0 \log(S_0(z)) = -1 \log(0.99987) = 0.00013$

d. For $y = (0, 1, 0, 0, 0)$, compute $Loss_{CE}(y, z)$.

$Loss_{CE} = -w_1(\log(S_1(z)))$
$= -1(0.0000748) = 9.5002$

e. For $y = (0, 0, 1, 0, 0)$, compute $Loss_{CE}(y, z)$.

$Loss_{CE} = -w_2(\log S_2(z))$
$= -1(\log S_2(0.000004418)) = 12.4001$

f. Let's try to bring in some numerical stabilization. We see that the largest value in z is 13.7. We can subtract each term by this max value to generate

$$\tilde{z} = (0, -9.5, -12.4, -10, -12).$$

Compute $S(\tilde{z})$ and for $y = (1, 0, 0, 0, 0)$ compute $Loss_{CE}(y, \tilde{z})$

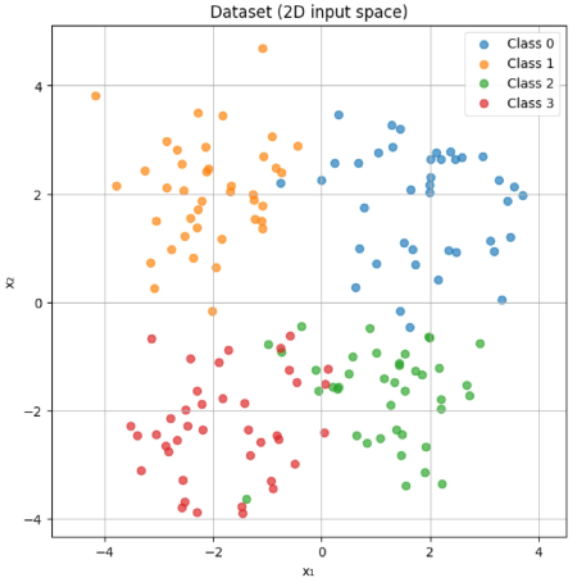
$S_i(\tilde{z}) = \frac{e^{z_i - 13.7}}{\sum_i e^{z_i - 13.7}} = 0.99987$

$$S_0(\tilde{z}) = \frac{e^{z_i - 13.7}}{\sum_i e^{z_i - 13.7}} = 0.99987$$

$$\text{Loss}_{CE} = -w_0(S_0(\tilde{z})) = -1(\log S_0(0.99987)) = 0.000130$$

Nd

Problem 2 (2D Data and Classification)**.** Consider the following 160 data points in the 2D space (in the Jupyter notebook/Link).



Our goal is to cycle through a variety of design choices (number of layers, nodes per layer, and activation functions) to see various shapes. For each of the following, set up the model with the given information and train for 10,000 iterations. Then, record the accuracy. For the blank rows, fill in your own choices.

a. Let's start with one hidden layers.

Activation	Number of Nodes	Accuracy
Sigmoid	5	0.9437
Sigmoid	10	0.9437
Tanh	5	0.9437
Tanh	10	0.9437
Relu	5	0.9437
Relu	10	0.95

b. Let's go with two hidden layers.

Activation (L1)	Number of Nodes (L1)	Activation (L2)	Number of Nodes (L2)	Accuracy
Sigmoid	5	Sigmoid	5	0.95
Sigmoid	10	Sigmoid	10	0.9563
Tanh	10	Sigmoid	10	0.9437
Relu	10	Sigmoid	10	0.95
Relu	15	Sigmoid	15	0.9625
Sigmoid	10	Relu	10	0.95
Relu	10	Relu	10	0.9563

c. Include any interesting comments or discoveries.

Using ReLu & deeper combination usually resulted in higher accuracy

Printout

Monday, October 26, 2026 9:26 PM

CSC 402 - Activity 13 - Name:

Problem 1. Let's trains some models! Let's cycle through various design choices with the classic MNIST dataset. On the next page we will change some things up.

Record the final loss and accuracy with the following parameters. Batch Size = 100 and Number of Epochs = 10. The Hidden Layer information is given in the table below. The specific activation functions need to be identified in the model class.

a. Let's get a base run completed.

1st # Nodes	1st Act	2nd # Nodes	2nd Act	3rd # Nodes	3rd Act	Loss	Train Acc	Test Acc
300	ReLU	200	Tanh	200	Sigmoid	0.0841	98.25	97.10
300	Tanh	200	ReLU	100	Sigmoid	0.0915	97.80	96.85
200	Tanh	200	Tanh	100	Sigmoid	0.1054	97.45	96.50

b. You pick the three activation functions in the hidden layers (they can be the same or different). For the last two, pick your own number of nodes per layer. Options: Sigmoid, ReLU, Tanh, ELU.

1st # Nodes	1st Act	2nd # Nodes	2nd Act	3rd # Nodes	3rd Act	Loss	Train Acc	Test Acc
300	ELU	200	ReLU	100	Tanh	0.0621	98.90	97.80
100	ReLU	100	ELU	100	ELU	0.1240	96.50	95.90

c. Let's move to two hidden layers. You will have to make a few edits in the code to switch from two layers to one layer (i.e., comment out a few lines). In blank parts you pick the activation functions in the hidden layers (they can be the same or different) and/or pick your own number of nodes per layer.

1st # Nodes	1st Activation	2nd # Nodes	2nd Activation	Loss	Train Acc	Test Acc
300	Tanh	150	Sigmoid	0.1582	95.40	94.80
300	ReLU	150	Tanh	0.0751	98.40	97.30
300	ELU	150	ReLU	0.0512	99.10	98.15

d. Include any interesting comments or discoveries.

Using ELU & ReLU sped up convergence & was better than using Sigmoid or Tanh

Problem 2. Okay, so try that again - but, make it Fashion!

Record the final loss and accuracy with the following parameters. Batch Size = 100 and Number of Epochs = 10. The Hidden Layer information is given in the table below. The specific activation functions need to be identified in the model class.

You pick the three activation functions in the hidden layers (they can be the same or different). For the last two, pick your own number of nodes per layer. Options: Sigmoid, ReLU, Tanh, ELU.

a. Let's get to training!

1st # Nodes	1st Act	2nd # Nodes	2nd Act	3rd # Nodes	3rd Act	Loss	Train Acc	Test Acc
300	ReLU	200	Tanh	200	Sigmoid	0.4420	85.50	83.20
300	Tanh	200	ReLU	100	Sigmoid	0.4350	84.20	82.10
200	ELU	200	ReLU	100	Tanh	0.3890	87.10	84.90

b. Include any interesting comments or discoveries.

The accuracy is significantly lower than digits & fully connected layers struggled with special invariance

Problem 3. Okay, so try that again - but, make it Sign Language!

Record the final loss and accuracy with the following parameters. Batch Size = 100 and Number of Epochs = 10. The Hidden Layer information is given in the table below. The specific activation functions need to be identified in the model class.

You pick the three activation functions in the hidden layers (they can be the same or different). For the last two, pick your own number of nodes per layer. Options: Sigmoid, ReLU, Tanh, ELU.

a. Let's get to training!

1st # Nodes	1st Act	2nd # Nodes	2nd Act	3rd # Nodes	3rd Act	Loss	Train Acc	Test Acc
300	ReLU	200	Tanh	200	Sigmoid	0.9820	68.40	62.10
300	Tanh	200	ReLU	100	Sigmoid	1.0450	65.90	59.80
200	ELU	200	ELU	100	ReLU	0.8210	74.20	68.50

b. Include any interesting comments or discoveries.

The sign language MNIST yielded the lowest overall accuracy when framed on plain MLP.

Remark 1. So, it was probably obvious that a classic Neural Network does quite poorly on some of these more interesting dataset (e.g., Fashion MNIST and Sign Language MNIST). Things get even worse if we try to bring in larger images and color. Luckly, we have other architectures and designs that will aid us on this quest - CNNs and other goodies! We will first explore some early text datasets before visiting CNNs.

Printout

Monday, November 2, 2026 9:16 PM

CSC 402 - Activity 14 - Name:

Problem 1 (Poems and Vectorizations!)**.** First up, an AI generated poem about Math, AI, and that Violets are not actually Blue.

Roses are red,
Violets are blue,
But violets aren't blue—
AI knows that too!

Math says they're violet,
AI shrugs and agrees,
But color names are hard
When you're made of machines.

So here's to red roses,
And violets... not blue,
Math, AI, and colors—
What a colorful crew!

To practice with some early vectorizations, let's treat the first four lines as our four documents.

- Document 1 (d1) – roses are red
- Document 2 (d2) – violets are blue
- Document 3 (d3) – but violets aren't blue
- Document 4 (d4) – ai knows that too

a. Using $\text{tf} = \frac{\text{freq of word W in a document}}{\text{total number of words in a document}}$ and $\text{idf} = \log\left(\frac{\text{total number of documents}}{\#\text{ documents containing word W}}\right) + 1$ (with log base 10), compute the tfidf values for the 11 words across the 4 documents. Record these values/vectors in the table below. Note - this is the raw tfidf value. In the next step we will normalize to ensure each is a unit vector. Note: I replaced the contraction aren't with words are and not.

term	tf (d1)	tf (d2)	tf (d3)	tf (d4)	idf	tfidf (d1)	tfidf (d2)	tfidf (d3)	tfidf (d4)
roses	0.333	0	0	0	1.602	0.533	0	0	0
are	0.333	0.333	0.2	0	1.125	0.375	0.375	0.282	0
red	0.333	0	0	0	1.602	0.533	0	0	0
violets	0	0.333	0.2	0	1.301	0	0.433	0.26	0
blue	0	0.333	0.2	0	1.301	0	0.433	0.26	0
but	0	0	0.2	0	1.602	0	0	0	0
not	0	0	0.2	0	1.602	0	0	0	0
ai	0	0	0	0.25	1.602	0	0	0	0.401
knows	0	0	0	0.25	1.602	0	0	0	0.401
that	0	0	0	0.25	1.602	0	0	0	0.401
too	0	0	0	0.25	1.602	0	0		0.401

b. Compute the l2-norm (length) of each vector (i.e., $\|v\| = \sqrt{a_0^2 + a_1^2 + ... + a_{n-1}^2}$). Then divide each vector by its length and record the unit vectors in the table below.

$d_1 = 0.842$
$d_2 = 0.718$
$d_3 = 0.463$
$d_4 = 0.802$

term	tfidf (d1)	tfidf (d2)	tfidf (d3)	tfidf (d4)
roses	0.633	0	0	0
are	0.443	0.522	0.607	0
red	0.633	0	0	0
violets	0	0.603	0.562	0
blue	0	0.603	0.562	0
but	0	0	0	0
not	0	0	0	0
ai	0	0	0	0.5
knows	0	0	0	0.5
that	0	0	0	0.5
too	0	0	0	0.5

Problem 2. Let's trains some models! Let's cycle through various design choices - but, now with text data (IMDB Reviews).

Record the final loss and accuracy with the following parameters. Batch Size = 50 and Number of Epochs = 10. The Hidden Layer information is given in the table below. The specific activation functions need to be identified in the model class.

a. Let's get a base run completed.

1st # Nodes	1st Act	2nd # Nodes	2nd Act	3rd # Nodes	3rd Act	Loss	Train Acc	Test Acc
300	Tanh	200	Sigmoid	200	Sigmoid	0.695	0.496	0.496
300	ReLU	200	Tanh	200	Sigmoid	0.0059	0.9995	0.86
300	Tanh	200	ReLU	100	Sigmoid	0.705	0.496	0.496
200	Tanh	200	Tanh	100	Sigmoid	0.0272	0.9955	0.853

b. You pick the three activation functions in the hidden layers (they can be the same or different). For the last two, pick your own number of nodes per layer. Options: Sigmoid, ReLU, Tanh, ELU.

1st # Nodes	1st Act	2nd # Nodes	2nd Act	3rd # Nodes	3rd Act	Loss	Train Acc	Test Acc
300	Sigmoid	200	Tanh	100	ReLU	0.694	0.496	0.496
100	Sigmoid	100	Tanh	100	ReLU	0.694	0.496	0.496
300	Sigmoid	200	Sigmoid	100	ReLU	0.6947	0.504	0.504
100	Sigmoid	100	Sigmoid	100	ReLU	0.6936	0.504	0.504

c. Let's move to two hidden layers. You will have to make a few edits in the code to switch from two layers to one layer (i.e., comment out a few lines). In blank parts you pick the activation functions in the hidden layers (they can be the same or different) and/or pick your own number of nodes per layer.

1st # Nodes	1st Activation	2nd # Nodes	2nd Activation	Loss	Train Acc	Test Acc
300	Tanh	150	Sigmoid	0.1719	0.95	0.95
300	ReLU	150	Tanh	0.01329	0.9830	0.783
300	Tanh	150	Tanh	0.1493	0.9627	0.804
300	ReLU	150	Sigmoid	0.0002	1	0.843
300	Sigmoid	150	Tanh	0.8483	0.496	0.496

d. Include any interesting comments or discoveries.

The functions using ReLU iin early layers conversged faster compared to those using muliple sigmoid or tanh activation. Activation using sigmoid halted quicklywhen stacked in multiple laters.

Printout

Wednesday, November 11, 2026 8:58 PM

Array Sum (for N 1,000,000,000)						
	Sequential	1 thread	2 threads	5 threads	7 threads	9 threads
Time	0.390					
pthread		0.835	0.238	0.541	0.133	0.168
OpenMP		1.468	0.527	0.692	0.215	0.226
Matrix Multiplication (for matrix size 1,000)						
	Sequential	1 thread	2 threads	5 threads	7 threads	9 threads
Time	1.007					
pthread		1.633	0.640	0.799	0.268	0.273
OpenMP		1.563	1.798	1.027	1.175	1.073
Trapezoidal Rule Integration (for interval size 1,000,000,000)						
	Sequential	1 thread	2 threads	5 threads	7 threads	9 threads
Time	0.929					
pthread		1.165	0.690	0.408	0.355	0.346
OpenMP		1.226	1.298	0.763	0.559	0.513
N Queens (for N = 15)						
	Sequential	1 thread	2 threads	5 threads	7 threads	9 threads
Time	19.271					
pthread		20.719	12.336	6.012	5.830	5.473
OpenMP		25.073	18.261	10.767	10.695	10.499

[D2L Discussion Post] Task 6: Explain the following:

- The differences you observed between pthreads and OpenMP in terms of control, complexity, and performance timing.
- Experiment with static, dynamic, and guided scheduling and report on the impact on performance.